

Aula 10: RNNs Avançadas, Seq2Seq e Embeddings

Treinamento, Geração de Sequências e Fatoração de Matrizes

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação (MDCC)
Universidade Federal do Ceará

Novembro de 2025

Roteiro da Aula

- 1 Recap: RNNs com Gates (LSTM e GRU)
- 2 Modelos com Gates: LSTM e GRU
- 3 Aspectos Práticos do Treinamento de RNNs
- 4 Modelagem Avançada: Processos de Ponto Neurais
- 5 Arquiteturas Seq2Seq e Geração
- 6 Embeddings de Palavras e Fatoração de Matrizes
- 7 Encerramento

Na Aula Anterior...

- Vimos que RNNs simples sofrem com o **desaparecimento e explosão de gradientes**, o que as impede de aprender dependências de longo prazo.
- A causa é a multiplicação sucessiva de matrizes Jacobianas na retropropagação: $\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$.
- Introduzimos as **LSTMs e GRUs** como uma solução arquitetônica para esse problema.
- A ideia central dessas arquiteturas é usar **portas (gates)** para controlar o fluxo de informação e, crucialmente, o fluxo de gradientes.

*Hoje, vamos entender em mais detalhes **como** os gates resolvem o problema do gradiente e discutir aspectos práticos do treinamento.*

A Regra da Produto da Probabilidade I

Decomposição

Qualquer distribuição de probabilidade conjunta sobre uma sequência $x_1, \dots, x_T$ pode ser decomposta em um produto de probabilidades condicionais:

$$\begin{aligned} p(x_1, \dots, x_N) &= p(x_1) \cdot p(x_2|x_1) \cdot \dots \cdot p(x_T|x_1, \dots, x_{T-1}) \\ &= \prod_{i=1}^T p(x_i|x_{1:i-1}) \\ &= \prod_{i=1}^T p(x_i|\mathcal{H}_i) \end{aligned}$$

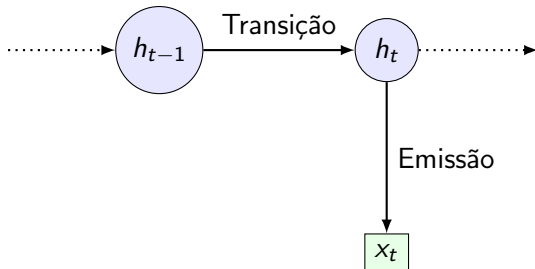
A Regra da Produto da Probabilidade II

- **O Problema:** O histórico $\mathcal{H}_t = x_{1:t-1} = \{x_1, \dots, x_{t-1}\}$ pode ser arbitrariamente longo. É impraticável estimar $p(x_i | \mathcal{H}_t)$ para todos os históricos possíveis.
- **A Solução:** Modelagem de $p(x_t | \mathcal{H}_t)$ utilizando um modelo de espaço de estado neural.

Modelos com Estado Latente I

A Ideia Central

Em vez de condicionar nos últimos $t - 1$ símbolos observados, vamos postular a existência de um **estado latente (ou oculto)** h_t que resume toda a informação necessária do passado.



A Ideia da Recorrência I

Estado Recorrente

Uma Rede Neural Recorrente (RNN) define o estado oculto h_t como uma função do estado anterior h_{t-1} e da entrada atual x_t .

$$h_t = f(h_{t-1}, x_t; \theta)$$

A forma mais simples (Elman RNN) é:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

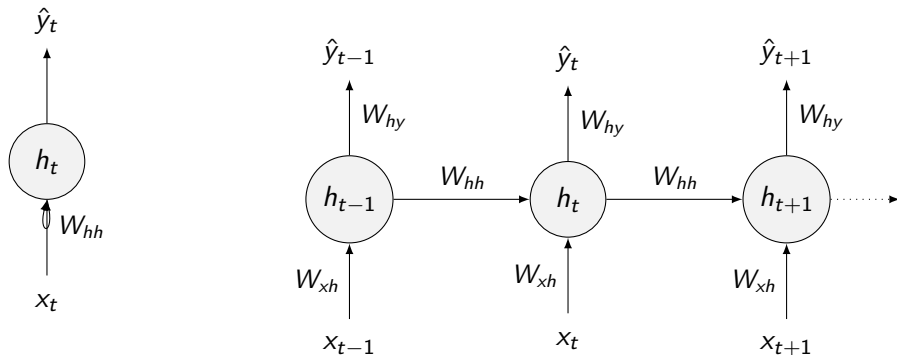
A Ideia da Recorrência II

Propriedade Fundamental

Os mesmos pesos (W_{hh} , W_{xh} , W_{hy}) e vieses (b_h , b_y) são **reutilizados em cada passo de tempo**. A rede tem uma forma de "memória" que é atualizada a cada passo, e os parâmetros que governam essa atualização são compartilhados ao longo de toda a sequência.

Visualização: Desdobramento no Tempo (Unrolling) I

Uma RNN pode ser vista como uma rede feed-forward muito profunda, onde cada camada corresponde a um passo de tempo e os pesos são compartilhados entre as camadas.



LSTM: Long Short-Term Memory I

Proposta por Hochreiter & Schmidhuber (1997), a LSTM introduz um **estado de célula** (C_t) que atua como uma "via expressa" para a informação, permitindo que ela flua sem grandes alterações por muitos passos. O fluxo é controlado por três "portões"(gates).

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Forget Gate: O que esquecer da célula})$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Input Gate: O que adicionar à célula?})$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (\text{Candidato a novo estado})$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{Atualização da célula})$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Output Gate: O que revelar?})$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{Novo estado oculto})$$

LSTM: Long Short-Term Memory II

onde σ é a função sigmoide e $\odot$ é a multiplicação elemento a elemento.

Gráfico Computacional da LSTM: Decomposição I

Entradas e Estado Interno

A célula LSTM opera em cada passo de tempo t recebendo o estado oculto anterior h_{t-1} e a entrada atual x_t . Ela mantém um **estado de célula** interno, C_{t-1} , que atua como sua memória de longo prazo. A computação é dividida em três estágios principais controlados por "gates".

Gráfico Computacional da LSTM: Decomposição II

1. Estágio de "Esquecimento" (Forget Gate Layer)

Primeiro, a célula decide qual informação do estado de célula anterior (C_{t-1}) deve ser descartada. A função sigmoide produz um número entre 0 ("esquecer completamente") e 1 ("manter completamente") para cada elemento do vetor C_{t-1} .

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Gráfico Computacional da LSTM: Decomposição III

2. Estágio de "Entrada/Armazenamento"(Input Gate Layer)

Em seguida, a célula decide quais novas informações serão armazenadas. Isso ocorre em duas partes:

- O "portão de entrada"(i_t) decide quais valores serão atualizados.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- Uma camada 'tanh' cria um vetor de novos valores candidatos, $\tilde{C}_t$, que poderiam ser adicionados ao estado.

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

O estado de célula é então atualizado: $C_t = (f_t \odot C_{t-1}) + (i_t \odot \tilde{C}_t)$

Gráfico Computacional da LSTM: Decomposição IV

3. Estágio de "Saída"(Output Gate Layer)

Finalmente, a célula decide qual será o próximo estado oculto h_t . O "portão de saída"(o_t) decide qual parte do estado de célula será revelada. O estado de célula é passado por uma 'tanh' e multiplicado pelo portão de saída.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

O Fluxo do Gradiente na LSTM I

A Chave: O Estado de Célula Aditivo

A equação de atualização do estado de célula da LSTM é:

$$C_t = \underbrace{f_t \odot C_{t-1}}_{\text{Esquecer o antigo}} + \underbrace{i_t \odot \tilde{C}_t}_{\text{Adicionar o novo}}$$

Vamos analisar a derivada de C_t em relação a C_{t-1} (ignorando o caminho do gradiente através de f_t e i_t por simplicidade):

$$\frac{\partial C_t}{\partial C_{t-1}} = f_t \odot 1 = f_t$$

O Fluxo do Gradiente na LSTM II

A "Via Expressa" do Gradiente

A retropropagação através da célula ao longo do tempo se torna:

$$\frac{\partial J}{\partial C_k} = \frac{\partial J}{\partial C_t} \left(\prod_{i=k+1}^t \frac{\partial C_i}{\partial C_{i-1}} \right) = \frac{\partial J}{\partial C_t} \left(\prod_{i=k+1}^t f_i \right)$$

- O gradiente não é mais atenuado por multiplicações repetidas pela matriz de pesos W_{hh} .
- O fluxo é controlado pelo **forget gate** (f_t), cujos valores estão entre 0 e 1.
- Se a rede aprender a manter $f_t \approx 1$, o gradiente pode fluir por muitos passos sem desaparecer. A rede pode **aprender a lembrar!**

GRU: Gated Recurrent Unit I

Uma Alternativa Simplificada à LSTM

Proposta por Cho et al. (2014), a GRU é uma variação da LSTM que visa o mesmo objetivo de combater o desaparecimento de gradientes, mas com uma arquitetura mais simples e menos parâmetros.

A GRU funde o estado de célula (C_t) e o estado oculto (h_t) em um único vetor h_t e usa apenas dois gates:

GRU: Gated Recurrent Unit II

Equações da GRU

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z) \quad (\text{Update Gate: novo estado})$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r) \quad (\text{Reset Gate: ignorar estado})$$

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t] + b_h) \quad (\text{Candidato a novo estado})$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{Novo estado oculto})$$

- O **Update Gate** (z_t) age como o forget e o input gate da LSTM. Ele decide o quanto manter do estado antigo ($1 - z_t$) e o quanto incorporar do novo estado candidato (z_t).

GRU: Gated Recurrent Unit III

- O **Reset Gate** (r_t) controla o quanto do estado anterior h_{t-1} é usado para calcular o novo candidato $\tilde{h}_t$. Se r_t for próximo de 0, a rede "esquece" o passado para calcular o novo estado.

O Fluxo do Gradiente na GRU I

Caminho Aditivo Controlado pelo Update Gate

Assim como a LSTM, a GRU também cria um caminho direto para o fluxo de gradientes. A equação de atualização é a chave:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Análise da Derivada

A derivada de h_t em relação a h_{t-1} possui um componente direto que não passa por uma multiplicação de matriz de pesos recorrente:

$$\frac{\partial h_t}{\partial h_{t-1}} = (1 - z_t) + \text{outros termos que dependem de } W$$

O Fluxo do Gradiente na GRU II

- O termo $(1 - z_t)$ funciona de forma análoga ao forget gate da LSTM. Ele cria uma "via expressa" para o gradiente.
- O **Update Gate** (z_t) aprende a controlar a força dessa conexão ao longo do tempo. Se a rede precisa lembrar de uma informação por um longo período, ela pode aprender a manter z_t próximo de 0, fazendo com que $(1 - z_t)$ seja próximo de 1.
- Isso permite que o gradiente flua por muitos passos sem desaparecer, resolvendo o mesmo problema fundamental que a LSTM, mas com menos complexidade.

O Fluxo do Gradiente na GRU III

LSTM vs. GRU

Não há um vencedor claro entre LSTM e GRU. A GRU tem menos parâmetros e pode treinar um pouco mais rápido. A LSTM, com seu estado de célula explícito, pode ser mais poderosa em tarefas que exigem uma contagem precisa ou o armazenamento de informações por períodos muito longos. Na prática, ambos são escolhas fortes e a seleção pode depender do problema e dos dados.

Inicialização e Regularização I

Inicialização de Pesos

Uma boa inicialização é crucial para a estabilidade do treinamento de RNNs.

- **Viés do Forget Gate:** Uma prática comum é inicializar o viés do forget gate (b_f) com um valor positivo (e.g., 1.0 ou 2.0). Isso encoraja o gate a ficar aberto no início do treinamento ($f_t \approx 1$), ajudando a preservar a memória e o fluxo do gradiente.
- **Matrizes de Pesos:** Inicialização ortogonal é frequentemente usada para as matrizes de peso, pois matrizes ortogonais preservam a norma dos vetores, ajudando a mitigar a explosão/desaparecimento de gradientes.

Inicialização e Regularização II

Regularização com Dropout

Aplicar dropout ingenuamente em conexões recorrentes ($h_{t-1} \rightarrow h_t$) é problemático, pois o ruído aleatório a cada passo pode destruir a capacidade da rede de manter memória.

- **Solução (Variational Dropout):** A mesma máscara de dropout é aplicada em todos os passos de tempo para uma determinada sequência. Isso regulariza a rede de forma consistente sem interferir na dinâmica recorrente.

Otimização para Sequências Longas I

Backpropagation Truncada Através do Tempo (TBPTT)

Fazer backpropagation sobre sequências muito longas (e.g., um documento inteiro) é computacionalmente caro e consome muita memória.

Otimização para Sequências Longas II

Heurísticas

- **A Solução:** A cada passo de otimização, desdobramos a rede e fazemos backpropagation por um número fixo de passos para trás (e.g., 30-50 passos), e não até o início da sequência.
- O estado oculto (h_t) continua sendo propagado para frente indefinidamente, mas o gradiente é "cortado" após um certo número de passos.
- É um compromisso: perdemos a capacidade de aprender dependências mais longas que a janela de truncamento, mas tornamos o treinamento viável.

Além de Timesteps Fixos: Processos de Ponto Temporais I

O Mundo é Assíncrono

Até agora, lidamos com sequências onde os eventos (e.g., palavras) ocorrem em passos discretos e regulares ($t = 1, 2, 3, \dots$). No entanto, muitos dados do mundo real são uma sequência de eventos que ocorrem em **tempo contínuo** e de forma irregular:

- Atividade em redes sociais (posts, likes).
- Transações financeiras.
- Terremotos e seus tremores secundários.
- Disparos de neurônios.

Esses dados são modelados por **Processos de Ponto Temporais (TPPs)**.

Além de Timesteps Fixos: Processos de Ponto Temporais II

A Função de Intensidade Condicional $\lambda(t|\mathcal{H}_t)$

Em vez de modelar $p(x_t|\mathcal{H}_t)$ para um evento discreto, um TPP modela a **taxa instantânea** de um evento ocorrer no tempo t , dado o histórico de eventos anteriores $\mathcal{H}_t = \{t_1, t_2, \dots, t_{i-1}\}$ com $t_i < t$.

$$\lambda(t|\mathcal{H}_t)dt = \mathbb{P}(\text{evento em } [t, t + dt]|\mathcal{H}_t)$$

A função de intensidade $\lambda(t|\mathcal{H}_t)$ nos diz a "probabilidade" de um evento acontecer "agora", e essa probabilidade muda dinamicamente com base no que já aconteceu.

Um Modelo Clássico: O Processo de Hawkes I

A Ideia: Auto-excitação

O Processo de Hawkes é um TPP clássico que modela o fenômeno de "clusterização" de eventos, onde a ocorrência de um evento aumenta a probabilidade de eventos futuros ocorrerem em seguida.

- Um tremor principal aumenta a chance de tremores secundários.
- Um post viral gera uma cascata de compartilhamentos.

Um Modelo Clássico: O Processo de Hawkes II

A Função de Intensidade do Processo de Hawkes

A intensidade em um processo de Hawkes tem uma forma específica:

$$\lambda(t) = \underbrace{\mu}_{\text{Intensidade base}} + \underbrace{\sum_{t_i < t} \kappa(t - t_i)}_{\text{Excitação de eventos passados}}$$

- μ : Uma taxa de fundo constante de eventos.
- $\kappa(\cdot)$: Uma "função kernel" que define como a influência de um evento passado decai com o tempo. Frequentemente, é um decaimento exponencial: $\kappa(\Delta t) = \alpha e^{-\beta \Delta t}$.

É um modelo poderoso, mas assume uma forma paramétrica fixa para a influência do histórico.

Aprendendo Processos de Ponto: A Log-Verossimilhança I

Como treinar um modelo de processo de ponto?

Para uma sequência de eventos $\{t_1, \dots, t_N\}$ observada em um intervalo $[0, T]$, a função de log-verossimilhança é composta por duas partes:

- 1 Maximizar a intensidade nos momentos em que os eventos ocorreram.** Queremos que $\lambda(t_i | \mathcal{H}_{t_i})$ seja alta para cada evento t_i .
- 2 Minimizar a intensidade nos momentos em que nada aconteceu.** A probabilidade de nenhum evento ocorrer no intervalo $[t, t + dt)$ é $1 - \lambda(t)dt \approx e^{-\lambda(t)dt}$.

Aprendendo Processos de Ponto: A Log-Verossimilhança II

A Função Objetivo

Combinando esses dois fatores, a log-verossimilhança a ser maximizada é:

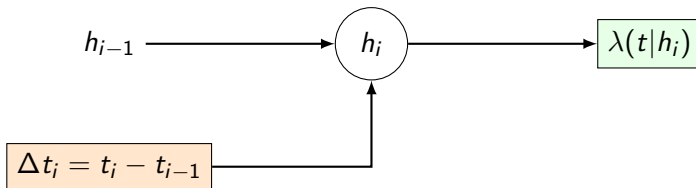
$$\mathcal{L}(\theta) = \underbrace{\sum_{i=1}^N \log \lambda(t_i | \mathcal{H}_{t_i}; \theta)}_{\text{Termo dos Eventos}} - \underbrace{\int_0^T \lambda(\tau | \mathcal{H}_{\tau}; \theta) d\tau}_{\text{Termo dos Não-Eventos}}$$

O desafio é que a integral pode ser difícil de calcular se $\lambda(t)$ for complexa.

Processos de Ponto Neurais: RNNs para a Intensidade I

A Grande Ideia: Generalizando o Processo de Hawkes

E se a influência do histórico na intensidade for mais complexa do que um simples decaimento exponencial? **Podemos usar uma RNN para aprender a função de intensidade a partir dos dados!** (Du et al., 2016)



Processos de Ponto Neurais: RNNs para a Intensidade II

Como Funciona

- 1** A RNN processa a sequência de eventos um por um. Em cada evento t_i , seu estado é atualizado. A entrada pode ser o tempo entre eventos, Δt_i .

$$h_i = f_{\text{RNN}}(h_{i-1}, \Delta t_i)$$

- 2** O estado oculto h_i se torna um resumo de todo o histórico $\mathcal{H}_{t_i}$.
- 3** A saída da RNN é usada para definir a função de intensidade $\lambda(t|h_i)$ para o próximo intervalo, até o próximo evento. Por exemplo, a RNN pode definir os parâmetros de uma função de intensidade mais simples:

$$\lambda(t|h_i) = \exp(w^\top h_i + b) \quad (\text{intensidade constante até } t_{i+1})$$

Recap: Arquitetura Encoder-Decoder I

O Paradigma Sequence-to-Sequence (Seq2Seq)

Mapeia uma sequência de entrada de comprimento S para uma sequência de saída de comprimento T .

- **Encoder:** Uma RNN processa a entrada e a comprime em um vetor de contexto c .
- **Decoder:** Outra RNN usa c para gerar a sequência de saída, um token de cada vez.

A tarefa é modelar $p(y_1, \dots, y_T | x_1, \dots, x_S)$.

Recap: Arquitetura Encoder-Decoder II

O Desafio da Geração (Inferência)

Durante o treinamento, conhecemos a sequência de saída correta. Mas, em tempo de teste, como geramos a melhor sequência de saída? Não podemos simplesmente enumerar todas as V^T possibilidades.

$$y^* = \arg \max_y p(y|x)$$

Esta busca é um problema intratável. Precisamos de algoritmos de busca aproximados.

Algoritmos de Decodificação (Decoding) I

Busca Gulosa (Greedy Search)

A abordagem mais simples: a cada passo t , escolher a palavra com a maior probabilidade.

$$y_t = \arg \max_{v \in \mathcal{V}} p(v|y_{<t}, c)$$

- **Prós:** Muito rápido e simples de implementar.
- **Contras:** Míope. Uma escolha localmente ótima pode levar a uma sequência globalmente sub-ótima. Pode produzir frases repetitivas ou incoerentes.

Algoritmos de Decodificação (Decoding) II

Busca em Feixe (Beam Search)

Uma heurística que explora um espaço de busca maior. Mantém um conjunto (o "feixe") das k hipóteses parciais mais prováveis a cada passo.

- 1 No passo t , para cada uma das k hipóteses em B_{t-1} , gere todas as V continuações possíveis.
- 2 Calcule a probabilidade de todas as $k \times V$ novas hipóteses.
- 3 Mantenha apenas as k melhores para formar o novo feixe B_t .
- 4 Continue até que todas as hipóteses no feixe gerem um token de fim ('<EOS>').

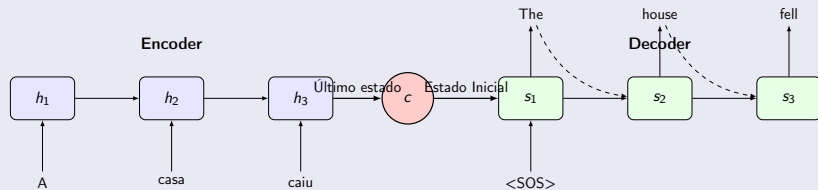
O parâmetro k é o **tamanho do feixe (beam size)**. $k = 1$ é a busca gulosa.

Arquitetura Seq2Seq: O Modelo Clássico I

O Paradigma Encoder-Decoder (Sutskever et al., 2014)

O modelo original de Seq2Seq usa o último estado oculto do encoder como o estado inicial do decoder. O decoder, então, opera de forma autônoma, usando apenas esse estado inicial e as palavras que ele mesmo gerou.

Diagrama



Arquitetura Seq2Seq: O Modelo Clássico II

O Gargalo de Informação

Toda a informação da sequência de entrada deve ser comprimida no único vetor de contexto c . Para sequências longas, isso é um gargalo significativo e a rede pode "esquecer" informações do início da entrada.

Arquitetura Seq2Seq: Contexto a Cada Passo I

Melhorando o Fluxo de Informação (Cho et al., 2014)

Uma melhoria fundamental é fornecer o vetor de contexto c como entrada para o decoder a **cada passo de tempo**, além de usá-lo para inicializar o estado. Isso dá ao decoder acesso constante à representação da sequência de entrada.

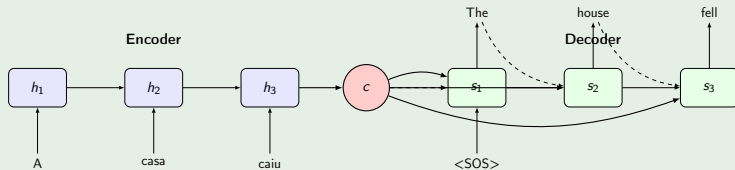
Arquitetura Seq2Seq: Contexto a Cada Passo II

Implementação

Na prática, o vetor de contexto c é concatenado com a entrada do decoder a cada passo. A equação de atualização do decoder se torna:

$$s_t = f(s_{t-1}, [y_{t-1}, c])$$

Isso alivia o problema do gargalo, mas ainda força uma única representação estática para toda a entrada.



Revisitando os Embeddings de Palavras I

A Ideia Central

Representar palavras como vetores densos em um espaço de baixa dimensão, onde a geometria do espaço captura relações semânticas e sintáticas.

- Ex: $\text{vec}(\text{rei}) - \text{vec}(\text{homem}) + \text{vec}(\text{mulher}) \approx \text{vec}(\text{rainha})$

Revisitando os Embeddings de Palavras II

Como são aprendidos?

Até agora, vimos os embeddings como uma camada de entrada (M) que é aprendida junto com o resto do modelo para uma tarefa específica (e.g., modelagem de linguagem).

No entanto, eles podem ser **pré-treinados** em um corpus massivo de texto não rotulado com objetivos específicos. O mais famoso é o **Word2Vec** (Mikolov et al., 2013).

Word2Vec e a Matriz de Coocorrência I

A Hipótese Distribucional

"Uma palavra é caracterizada pela companhia que ela mantém"(J.R. Firth).

- A base para aprender embeddings é contar com que frequência as palavras aparecem juntas (coocorrem) em um contexto.
- Podemos construir uma matriz de coocorrência **Palavra-Contexto**, X , onde X_{ij} é uma estatística de quão frequentemente a palavra i aparece no contexto da palavra j .
- Essa matriz é gigante ($|\mathcal{V}| \times |\mathcal{V}|$), esparsa e ruidosa.

Word2Vec e a Matriz de Coocorrência II

Exemplo de Matriz X (Contagens Brutas)

	gato	cachorro	sentou	tapete
gato	0	5	12	3
cachorro	5	0	9	2
sentou	12	9	0	20
tapete	3	2	20	0

A Conexão: Embeddings como Fatoração de Matrizes I

Redução de Dimensionalidade

A matriz de coocorrência X é uma representação de alta dimensão. Queremos uma representação de baixa dimensão. A **Fatoração de Matrizes** é uma técnica clássica para isso.

$$X_{|\mathcal{V}| \times |\mathcal{V}|} \approx W_{|\mathcal{V}| \times d} \cdot C_{d \times |\mathcal{V}|}^T$$

- W : Matriz de embeddings de palavras de baixa dimensão ($d \ll |\mathcal{V}|$).
- C : Matriz de embeddings de contexto de baixa dimensão.

A Conexão: Embeddings como Fatoração de Matrizes II

O Insight de Levy & Goldberg (2014)

Treinar o modelo **Word2Vec (Skip-gram com amostragem negativa)** é matematicamente equivalente a fatorar implicitamente uma matriz de coocorrência!

- A matriz que está sendo fatorada não é de contagens brutas, mas sim a matriz de **Pointwise Mutual Information (PMI)** deslocada.
- $PMI(w, c) = \log \frac{P(w, c)}{P(w)P(c)}$. Mede se duas palavras coocorrem mais do que o esperado por acaso.

Isso unifica duas grandes áreas de pesquisa: a aprendizagem de representações neurais e a semântica distribucional baseada em contagens.

O Que Significa o Vetor de Contexto (c)? I

A Ideia de um "Thought Vector"

O vetor de contexto c é a peça central da arquitetura encoder-decoder. A sua intenção é ser um **resumo numérico** de toda a sequência de entrada.

Idealmente, c deveria capturar a **semântica**, a **sintaxe** e a **intenção** da frase original em um único ponto no espaço vetorial. Por exemplo, para a entrada "A casa caiu", o encoder deveria produzir um vetor c que representa o conceito de "colapso de uma casa".

O Que Significa o Vetor de Contexto (c)? II

Características do Contexto Estático

Nos modelos que acabamos de ver, este vetor de contexto é:

- **Holístico:** Tenta encapsular o significado da sequência *inteira* em uma única representação.
- **Estático:** O mesmo vetor c é usado para gerar cada palavra da sequência de saída. Ele não se adapta ao progresso da decodificação.
- **Um Gargalo:** É forçado a ser o único canal de informação entre o encoder e o decoder.

O Que Significa o Vetor de Contexto (c)? III

As Limitações Conceituais

Essa abordagem assume que um único vetor de tamanho fixo pode representar frases de qualquer comprimento, o que é problemático.

- **Perda de Informação:** Para frases longas, é inevitável que o encoder "esqueça" detalhes. Informações do início da frase têm menos chance de influenciar o estado final.
- **Falta de Foco:** Ao traduzir uma palavra, humanos focam em palavras específicas da fonte. O vetor c oferece um resumo "desfocado" da frase inteira, sem dar mais importância a nenhuma parte.

*Isso nos leva a uma pergunta fundamental: em vez de um único resumo estático, o decoder poderia aprender a **prestar atenção** a diferentes partes da entrada a cada passo?*

Resumo e Próximos Passos I

O que Vimos Hoje

- Aprofundamos o entendimento de como as **LSTMs** resolvem o problema do gradiente através de seu estado de célula aditivo.
- Discutimos dicas práticas para **treinar RNNs**, como inicialização, dropout e TBPTT.
- Analisamos como gerar sequências a partir de modelos **Seq2Seq** usando busca gulosa e busca em feixe.
- Revelamos a profunda conexão entre o aprendizado de **embeddings de palavras** e a **fatoração de matrizes** de coocorrência.

Referências I